

파이썬으로 생각표현하기

인공지능 개론

Jinu Gong

학습 목표

- 조건문의 필요성과 사용 방법에 대해 알아보자
- 반복문의 필요성과 사용 방법에 대해 알아보자
- 여러 개의 데이터를 하나로 묶어서 처리하는 리스트와 튜플을 배워보자
- 키를 이용해서 값을 추출할 수 있는 딕셔너리에 대하여 알아보자
- 자료값의 중복을 허용하지 않는 순서가 없는 자료형인 집합에 대하여 알아보자

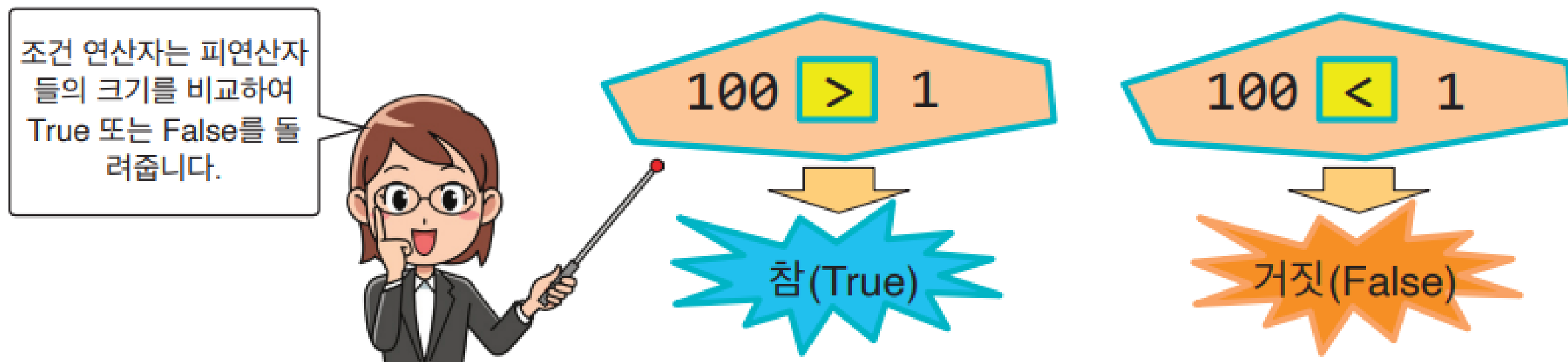
조건문

if 문

- 성적이 60점 이상이면 '합격입니다'를 출력하고, 60점 미만이면 '불합격입니다'를 출력한다고 하자. 이것은 성적이라는 데이터가 60점 이상인지 그렇지 않은지를 판단해야 한다.
- 이렇게 어떤 **조건**condition을 만족하는지 그렇지 않은지를 판정하는 식을 **조건식**condition expression이라고 한다. 그리고 이 조건식은 참 또는 거짓의 값을 갖는 **부울**bool형으로 평가된다.
- 앞에서 배운 **관계 연산자**relational operator는 두 개의 피연산자를 비교하는데 사용되는데, 관계 연산자의 결과는 **참**True 아니면 **거짓**False으로 나타난다. 따라서 이것은 참과 거짓의 값을 갖는 조건식이 될 수 있다.

if 문

- 파이썬에서 참과 거짓은 True와 False로 표시되기 때문에 True 또는 False가 관계 연산의 값으로 생성된다.
- **100 > 1** 라는 관계식을 예로 들어보자.
 - 100이 1보다 크기 때문에 이 수식은 참을 의미하는 값 True를 생성한다.
 - 반면 **1 > 100** 수식은 거짓이므로 False 값이 생성된다.
 - 조건식은 이렇게 참과 거짓의 값을 가질 수 있는 수식을 의미한다.



if 문

- if를 이용한 간단한 선택 구조를 만들어 보자. if 문은 다음과 같은 문법구조를 갖는다. 아래 문법 에 나타나 있는 조건식이란 True, False를 가질 수 있는 식을 말한다.

```
if 조건식 :  
    실행할 내용
```

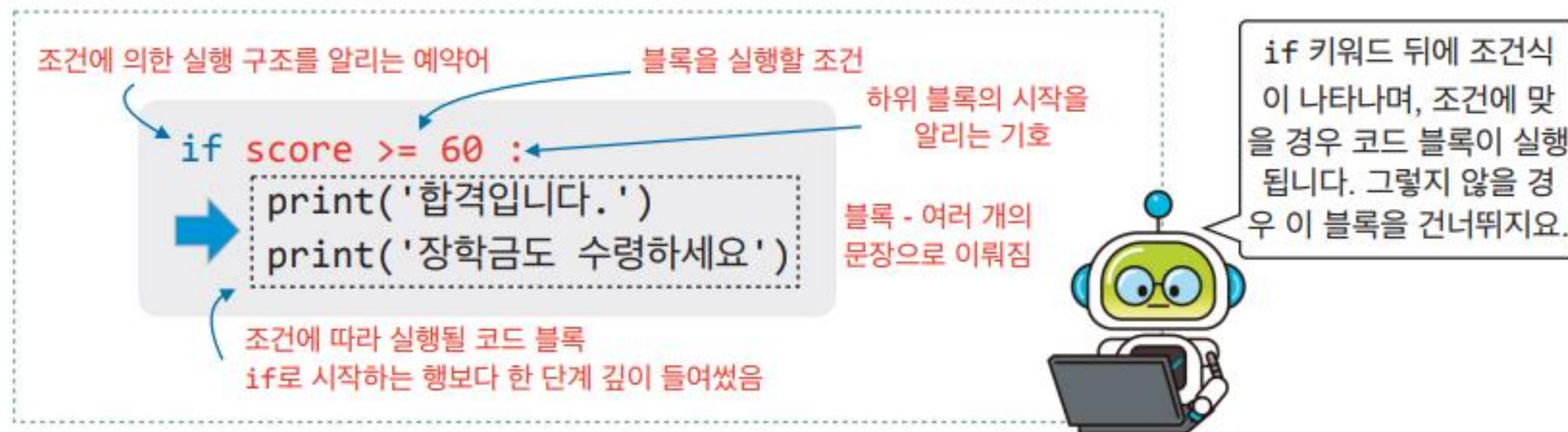
- 다음과 같이 변수 x에 100을 넣어준 후 아래와 같이 if 문을 작성해 보자. 이 경우 조건식에 해당하는 $x > 1$ 이 True를 반환하므로 실행할 내용이 선택되어 실행
 - 화면에는 '**x는 1보다 큼니다.**'가 출력되는 것을 볼 수 있다.
 - 이때 :(콜론) 아래의 줄은 **반드시 들여쓰기**를 해야만 한다.

```
x = 100  
if x > 1:  
    print("x는 1보다 큼니다.")
```

```
x는 1보다 큼니다.
```

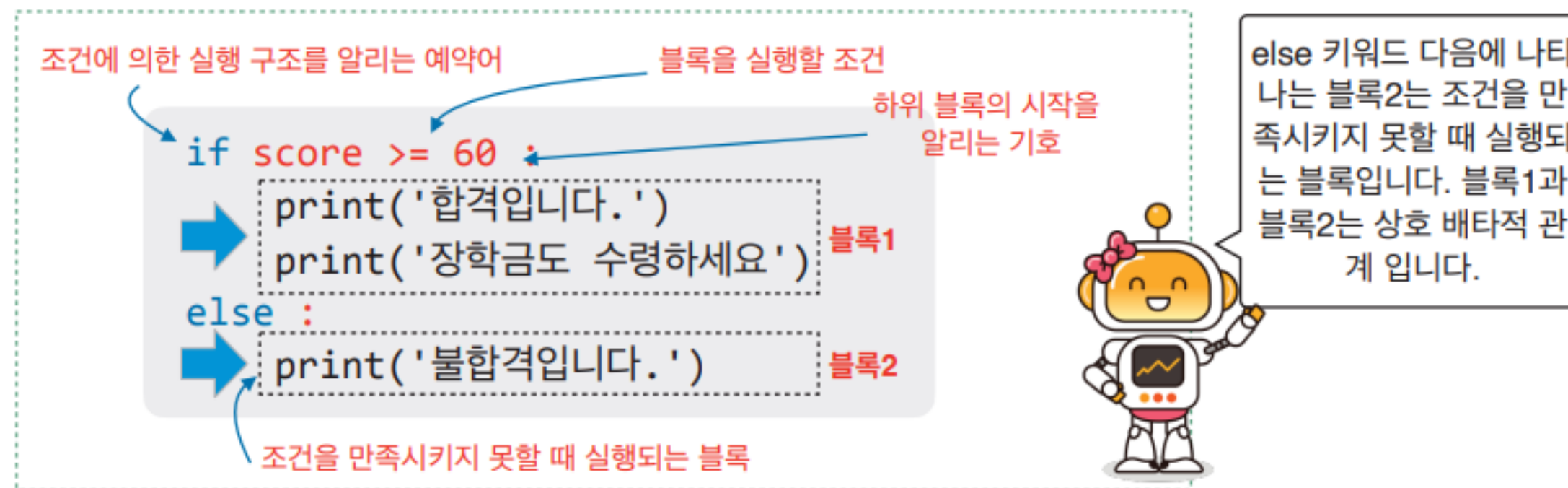
if 문

- 블록은 들여쓰기로 완성
 - 성적이 60점 이상이면 합격과 동시에 장학금도 받을 수 있다고 출력하려면 어떻게 해야 할까?
 - 다음과 같이 들여쓰기를 이용하여서 문장들을 묶을 수 있다.
 - 아래의 코드에서 score의 값이 60이상이면 print()를 호출하는 2개의 문장이 실행된다.
 - 이들 문장들이 동일한 개수의 공백을 가지고 있다는 것에 유의
 - 이들 모두는 동일한 **블록**block에 속해 있다.



if-else 문

- 파이썬에서는 선택 구조를 위하여 if-else 문을 제공한다. 성적이 60점 이상이면 합격으로, 60점 미만이면 불합격으로 처리해야 한다고 하자. 파이썬으로 다음과 같이 작성할 수 있다.



- if-else 문장은 "만약 조건이 참이면 이것을 실행하고, 조건이 참이 아니면 저것을 실행해!"라고 말하는 것과 같다. if-else 문에서는 조건을 수식으로 표현하며, 그 수식을 바로 '조건식' 이라고 한다.

if-else 문

- 조건식 뒤에는 콜론(:)이 있다. 콜론(:)은 파이썬 인터프리터에게 "아직 전체 문장을 끝나지 않았으니 잠시 해석을 미뤄 달라. 이 다음에는 이 문장에 딸린 코드 블록이 나타날 것이다."라고 요청하는 기호이다.

```
if score >= 60:  
    print("합격입니다.")  
else:  
    print("불합격입니다.")
```

if-else 문

- 또 다른 예시로 24시 체계의 시간정보를 입력 받아 12보다 작으면 '오전입니다.' 12이상이면 '오후입니다.'를 출력하는 시계가 있다고 가정하자. 0에서 23까지의 시간이 입력될 경우 오전이면서 동시에 오후가 될 수 없으므로 이 조건은 **배타적 조건**exclusive condition이 되어 다음과 같은 if-else 문으로 표현 가능하다.

```
hour = 10

if hour < 12:
    print('오전입니다.')
else:
    print('오후입니다.')
```

if-else 문

- 사용자로부터 성적을 입력받아서 합격 여부를 판단해 보자. 아래 코드처럼 사용자의 성적을 입력받는다. 입력된 값이 60 이상이면 프로그램은 '합격입니다.'를 출력하고, 그렇지 않으면 '불합격입니다.'를 출력한다.

```
score = int(input("성적을 입력하시오: "))  
if score >= 60:  
    print("합격입니다.")  
else:  
    print("불합격입니다.")
```

```
성적을 입력하시오: 80  
합격입니다.
```

if-else 문

- 사용자로부터 자연수를 입력받아서 짝수인지 홀수인지를 검사하는 프로그램을 작성해보자.

```
num = int(input("양의 정수를 입력하시오: "))  
if num % 2 == 0 :  
    print("짝수입니다.")  
else:  
    print("홀수입니다.")
```

```
양의 정수를 입력하시오: 10  
짝수입니다.
```

if-else 문

- 만일 임의의 정수를 입력 받은 후 이 검사를 0 이상인 수에 대해서만 하고 음수인 경우에 대해서는 '음수입니다'를 출력하는 **좀 더 복잡한 조건을 설정**하려면 어떻게 할까?
- 이 경우에는 다음처럼 if-else문 내에 또다시 if-else 조건을 달아서 해결할 수 있다. 이러한 경우 조건문 내의 조건문을 내부 조건문 이라고 한다.

```
num = int(input("정수를 입력하시오: "))
```

```
if num < 0 :
```

```
    print("음수입니다.")
```

```
else:
```

```
    print("양수입니다.")
```

```
    if num % 2 == 0 :
```

```
        print("짝수입니다.")
```

```
    else:
```

```
        print("홀수입니다.")
```

음수 vs 양수

짝수 vs 홀수

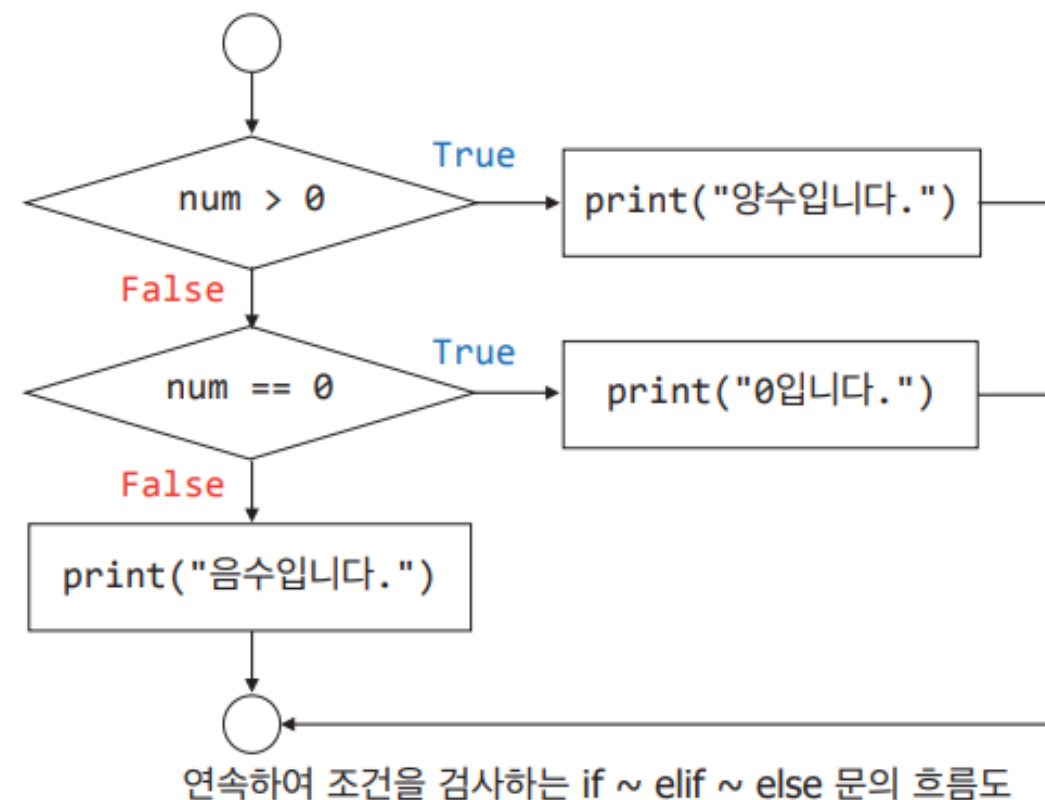
```
정수를 입력하시오: 10
```

```
양수입니다.
```

```
짝수입니다.
```


elif

- if-else 문에서 조건이 거짓일 때 다른 조건을 검사할 수 있다. 이때는 elif 키워드를 사용하면 된다.
 - elif는 "else if"를 합친 것이다.
- 사용자로부터 정수 num를 입력받아서 이 값을 보고 "양수입니다.", "0입니다.", "음수입니다."를 출력하는 프로그램을 고려해보자.



elif

- 이와 같은 구조의 프로그램은 if-elif-else를 사용하여 다음과 같이 구현할 수 있다.

```
num = int(input("정수를 입력하시오: "))    # 사용자의 입력을 받아서 정수값으로 변환함

if num > 0:                                # 첫 번째 조건을 만족하면 두번째 elif 문의 조건을 검사하지 않음
    print("양수입니다.")
elif num == 0:                             # 첫 번째 조건을 만족하지 못할 경우 조건 검사를 수행함
    print("0입니다.")
else:                                       # 위의 두 조건을 모두 만족시키지 못한 경우 수행되는 문장
    print("음수입니다.")
```

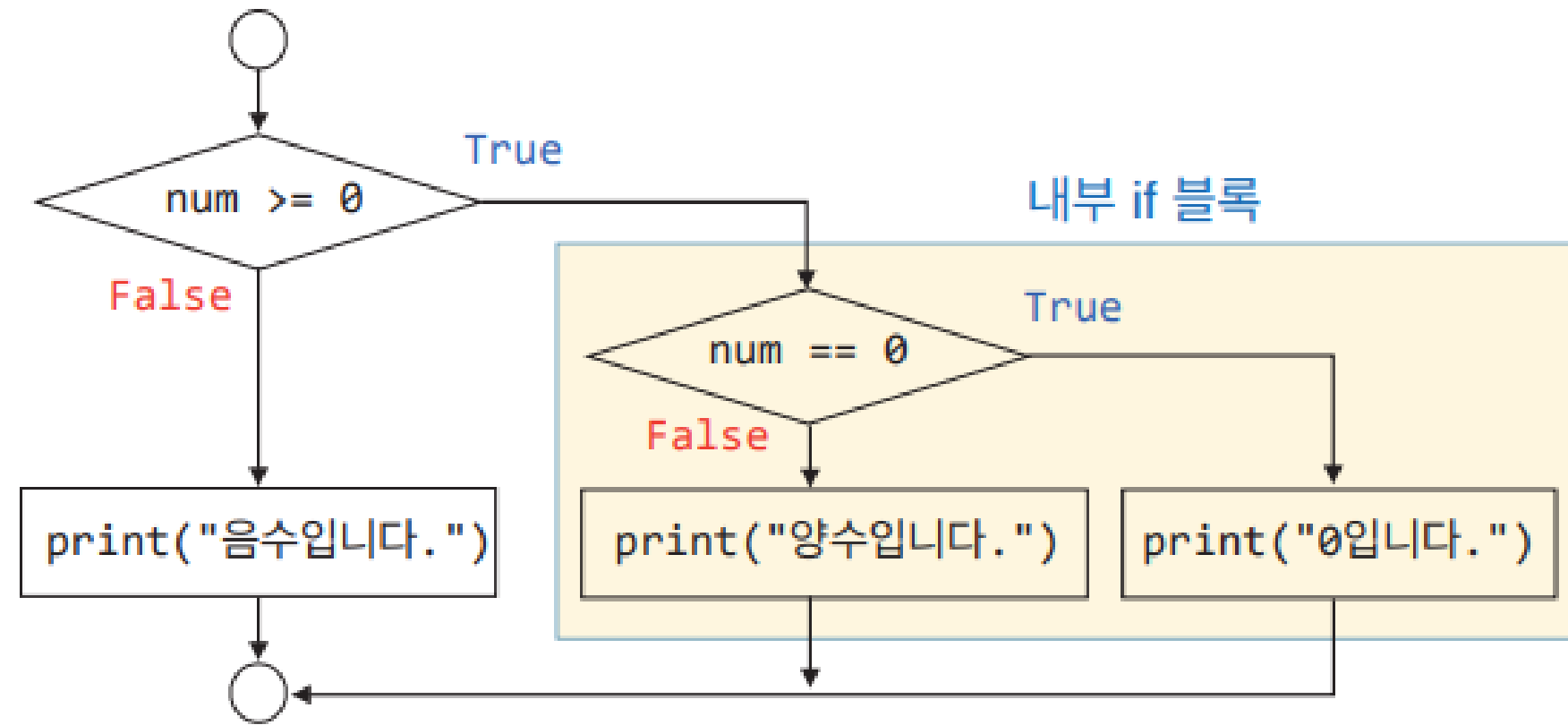
```
정수를 입력하시오: 0
0입니다.
```

```
정수를 입력하시오: 10
양수입니다.
```

```
정수를 입력하시오: -10
음수입니다.
```

내부 if 블록

- 필요에 따라 if 문 안에 또 다른 if 문이 들어갈 수도 있는데, 이것을 **내부 if 블록**, 또는 **중첩 if 블록**이라고 한다.
 - 이것은 다음과 같이 특정한 상황에서 True라는 조건이 나왔을 경우, 이 상황에 대해서 또 다른 세부적인 조건을 검사하는 상황에서 발생할 수 있다.



if 문과 내부 if 블록을 사용하여 조건을 검사하는 프로그램의 흐름도

내부 if 블록

- 이 예제에서는 사용자로부터 num을 입력받은 후 이 값이 $\text{num} \geq 0$ 일 경우 num이 0인지 아닌지 또 다른 조건 검사를 하여 이에 따라 "0입니다."와 "양수입니다."를 출력한다.
- 이제 다음과 같이 사용자에게서 정수를 받아서 양수인지, 0인지, 음수인지를 구별하여 화면에 출력하여 보자.

```
num = int(input("정수를 입력하시오: "))
if num >= 0:                                # if 문으로 조건을 검사하고 반드시 들여쓰기를 하여 블록을 생성
    if num == 0:                            # 블록 내에서 세부적인 조건을 한 번 더 검사
        print("0입니다.")                 # 또 다시 들여쓰기를 할 것
    else:
        print("양수입니다.")
else:
    print("음수입니다.")
```

```
정수를 입력하시오: 0
0입니다.
정수를 입력하시오: 10
양수입니다.
정수를 입력하시오: -10
음수입니다.
```



반복문

반복문

- 왜 반복문이 중요할까?
 - 반복은 어떤 단계를 반복하게 하는 것으로 반복 구조를 사용하면 프로그램이 간단하고 빠르게 된다.
 - 예를 들어서 동일한 작업을 반복하기 위하여 똑같은 문장을 복사하여 붙여넣기 하는 것보다 반복 구조를 사용하는 편이 프로그램을 간결하게 만든다.
 - 또 프로그래밍에 필요한 시간도 단축할 수 있다.

반복문

- 왜 반복문이 중요할까?

- 하나의 예로 화면에 회사에 중요한 손님이 오셔서 대형 전광판에 다음과 같이 환영인사를 5번 출력한다고 하자.

```
파이썬 주식회사의 방문을 환영합니다!  
파이썬 주식회사의 방문을 환영합니다!  
파이썬 주식회사의 방문을 환영합니다!  
파이썬 주식회사의 방문을 환영합니다!  
파이썬 주식회사의 방문을 환영합니다!
```

- 반복 구조를 사용하지 않는다면 다음과 같이 동일한 문장을 Ctrl+C를 사용하여 복사한 후에 Ctrl+V로 붙여넣기 하여야 한다.

```
print("파이썬 주식회사의 방문을 환영합니다!")  
print("파이썬 주식회사의 방문을 환영합니다!")  
print("파이썬 주식회사의 방문을 환영합니다!")  
print("파이썬 주식회사의 방문을 환영합니다!")  
print("파이썬 주식회사의 방문을 환영합니다!")
```

반복문

- 왜 반복문이 중요할까?
 - 물론 반복 횟수가 몇 번 안 되는 경우에는 위와 같이 “복사해서 붙여넣기” 하여도 된다.
 - 하지만 100번 반복해야 한다면 어떻게 할 것인가?
 - 이런 경우에 for 문을 사용한다면 다음과 같이 간단하게 작성할 수 있다.

```
for i in range(100):  
    print("파이썬 주식회사의 방문을 환영합니다!")
```

- 이와 같이 반복문은 인간이 **반복적으로 수행하는 번거로운 작업을 손쉽게 해결해** 줄 수 있으므로 매우 중요한 제어문으로 널리 사용되고 있다.

반복문

- 반복의 종류

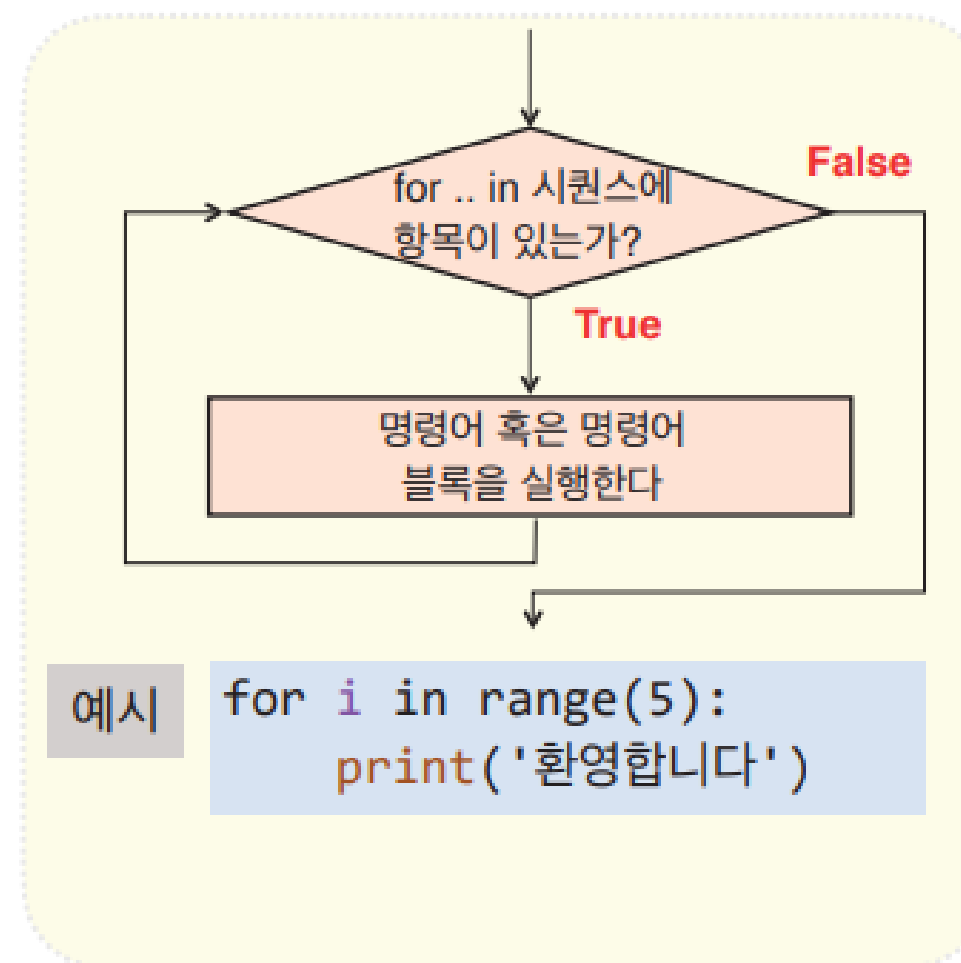
- 파이썬에서는 2 가지 종류의 반복이 있다.
 1. 횟수 제어 반복(**for 문**): 보통 횟수를 정해 놓고 반복한다.
 2. 조건 제어 반복(**while 문**): 특정한 조건이 만족되면 계속 반복한다.
- 횟수 제어 반복은 반복을 시작하기 전에 반복의 횟수를 미리 아는 경우에 사용한다.
- 예를 들어서 “환영합니다” 문장을 5번 반복하여 출력한다면 for 문을 사용하면 자연스럽다.
- 파이썬에서는 항목들을 모아 놓은 시퀀스라는 객체가 있고 여기에서 항목을 하나씩 가져와서 반복 할 때도 **for** 문이 적합하다. 시퀀스에 항목이 더 이상 없으면 반복이 종료된다.

반복문

- For 문

- range(5)라는 함수는 0, 1, 2, 3, 4의 숫자 시퀀스를 자동으로 생성하는 함수
- 따라서 이 문장은 0에서 4까지의 숫자를 반환하는 작업을 끝낼 때 까지 반복 실행

1) for 문

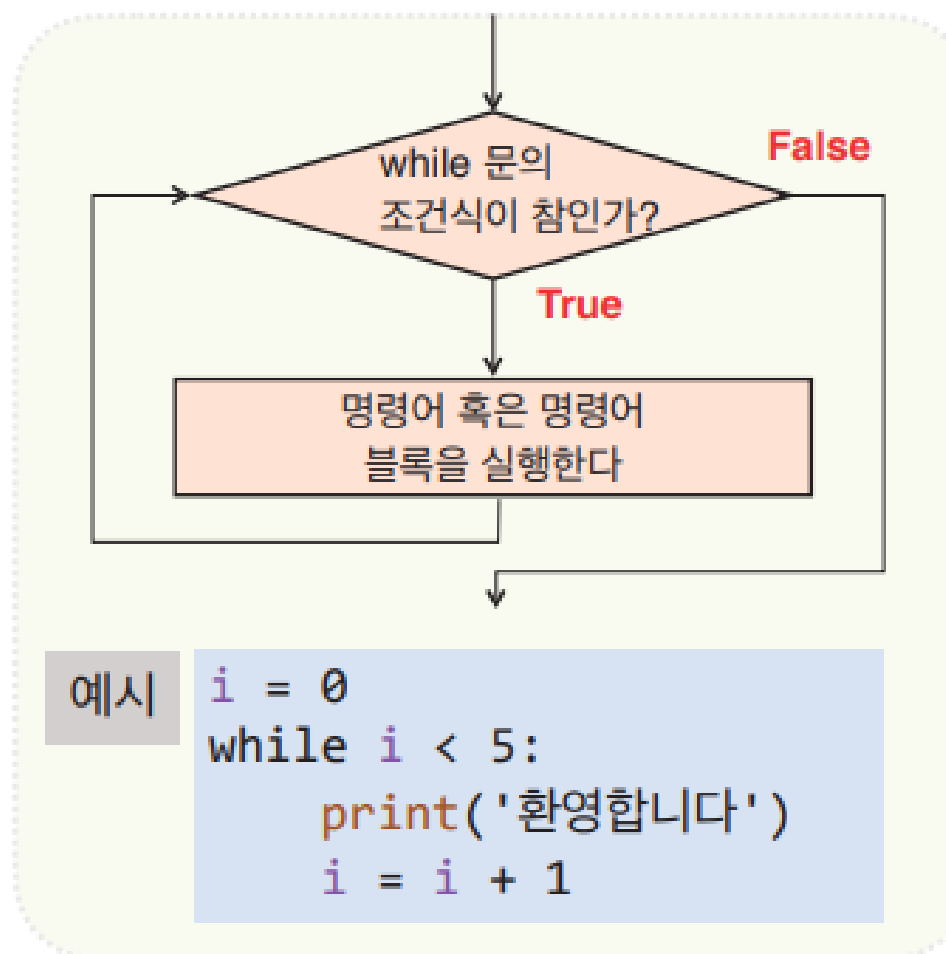


반복문

- While 문

- while문은 $i < 5$ 의 조건이 참일 경우 '환영합니다'를 출력하고 i 값을 1 증가
- 최초의 i 값이 0이므로 이 반복문 역시 5회 반복 수행

2) while 문



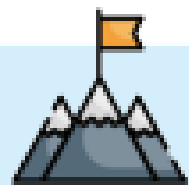
반복문

• 반복의 종류



잠깐 - 루프^{loop}

프로그래밍에서 반복은 흔히 올라미 같은 동그란 고리를 의미하는 **루프^{loop}**라고 한다. 왜냐하면 프로그램이 반복할 때 이전 단계로 되돌아가는데 이것이 동그라미를 그리는 것처럼 보이기 때문이다. 이 고리를 빠져나오지 않고 계속해서 반복을 하는 경우를 **무한루프^{infinite loop}**라고 한다.



도전문제 5.2

- (1) `for t in range(10):`를 이용하여 블록 내부에서 `print()` 함수를 사용하여 `t` 값을 출력해 보자.
- (2) `for t in range(1, 10):`를 이용하여 블록 내부에서 `print()` 함수를 사용하여 `t` 값을 출력해 보자.
- (3) `for t in range(2, 10):`를 이용하여 블록 내부에서 `print()` 함수를 사용하여 `t` 값을 출력해 보자.
- (4) `for t in range(2, 10, 2):`를 이용하여 블록 내부에서 `print()` 함수를 사용하여 `t` 값을 출력해 보자.
- (5) `for t in range(2, 10, 3):`를 이용하여 블록 내부에서 `print()` 함수를 사용하여 `t` 값을 출력해 보자.

for 루프

- 파이썬에서 횟수 제어 반복은 for 루프를 이용하는 것이 가장 간편하다. for 루프는 반복할 횟수를 정해 두고, 이 횟수가 만족될 때까지 같은 동작을 반복하도록 하는 일을 쉽게 표현할 수 있다.



for 루프

- for 루프를 사용할 때는 다음과 같이 리스트를 이용하여 사용하는 방식을 먼저 연습해 보자. 아래와 같은 코드로 반복을 할 수 있다. 여기서 [...]은 **리스트 list** 자료형이다.
- 이것은 7장에서 더욱 자세히 다룰 것인데, 여러 개의 값들을 담을 수 있는 장바구니와 같은 개념으로 이해하면 된다. 리스트 [1, 2, 3, 4, 5] 안에는 정수 1, 2, 3, 4, 5가 담겨있다고 생각하면 된다.

```
for i in [1, 2, 3, 4, 5]:    # 반복문의 끝에 :이 있어야 함
    print("방문을 환영합니다!") # if 문과 마찬가지로 들여쓰기를 하여야 함
```

for 루프

- 반복해서 실행될 문장은 반드시 들여쓰기를 하여야 한다.
- for 문에 담긴 **블록**block이라 생각하면 된다. 위의 코드를 실행하면 다음과 같은 출력을 얻을 수 있다.

```
방문을 환영합니다!  
방문을 환영합니다!  
방문을 환영합니다!  
방문을 환영합니다!  
방문을 환영합니다!
```

- 첫 번째 반복에서 변수 `i`의 값은 리스트의 첫 번째 숫자인 1이 되고 `print(...)` 문장이 실행된다.
- 두 번째 반복에서 변수 `i`의 값은 리스트의 두 번째 숫자인 2가 되고 `print(...)` 문장이 실행된다.
- 세 번째 반복에서 변수 `i`의 값은 리스트의 세 번째 숫자인 3이 되고 `print(...)` 문장이 실행된다.
- 네 번째 반복에서 변수 `i`의 값은 리스트의 네 번째 숫자인 4가 되고 `print(...)` 문장이 실행된다.
- 다섯 번째 반복에서 변수 `i`의 값은 리스트의 다섯 번째 숫자인 5가 되고 `print(...)` 문장이 실행된다.

for 루프

- for – in

- 앞 페이지에서는 변수 `i`의 값을 전혀 이용하지 않았다! 이번에는 반복하면서 `print()` 함수를 이용해서 변수 `i`의 값을 출력하여 보자.

```
for i in [1, 2, 3, 4, 5]:      # in 뒤에 리스트를 넣고 끝에 :을 넣자
    print("i =", i)          # i 값을 출력해보자
```

- 위의 코드를 실행하면 다음과 같이 출력된다.

```
i = 1
i = 2
i = 3
i = 4
i = 5
```


for 루프

- for – in

- 이 문장은 다음과 같은 문자열에 대해서도 적용할 수 있다.

```
for i in "Hello":      # 끝에 :이 있어야 함
    print("i =", i)    # i 값을 출력해보자
```

```
i = H
i = e
i = l
i = l
i = o
```

for 루프

- for – in

- 이번에는 이러한 출력 방식을 조금 더 개선해서 다음과 같이 구구단 9단 중의 일부를 for 루프를 이용하여 출력하여 보자.

```
for i in [1, 2, 3, 4, 5]:           # 1에서 5까지의 리스트
    print("9 *", i, "=", 9 * i )    # 9*i 값을 출력해보자
```

```
9 * 1 = 9
9 * 2 = 18
9 * 3 = 27
9 * 4 = 36
9 * 5 = 45
```

range() 함수

- 앞에서는 리스트에 정수들을 저장해두고 하나씩 꺼내서 반복하였다.
- 하지만 반복 횟수가 1000번이라면 이 방법이 불가능하다.
- 반복해야 하는 횟수가 큰 경우를 대비하여 range() 함수를 사용하는 방법이 준비되어 있다.
- range() 함수로 반복 횟수를 전달하면 range() 함수가 자동으로 정수들을 생성해준다.



range() 함수

- range(3) 함수는 0, 1, 2까지의 **숫자열sequence**을 반환한다. 반복할 때마다 변수 i에 이 값들을 대입하면서 문장을 반복한다. 즉, 첫 번째 반복에서는 i는 0이고 되고 두 번째 반복에서는 1이 된다. 마지막 반복에서 i는 2가 된다.
- 하나 이상의 반복되는 문장이 올 수 있는데, 반복되는 문장은 if 문과 같이 **반드시 동일한 간격의 들여쓰기를 해야 한다**. 이 때 들여쓰기가 있는 문장들만이 반복된다.
- 파이썬 셸에서 range() 함수에 list() 함수를 적용시키면 range() 함수가 생성하는 정수들을 볼 수 있다. 이와 같이 range() 함수는 연속적인 값들을 생성하는 일을 한다.

```
>>> list(range(10))  
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```



잠깐 – range() 함수가 반환하는 값은 range 형이다

파이썬의 range() 함수는 range 형의 자료형을 반환한다. range 형의 자료형은 호출이 발생할 때마다 매번 연속된 값을 생성하여 반환하는 특별한 일을 한다. 따라서 주로 for 문과 함께 사용된다.

range() 함수

- range() 함수는 여러 개의 인자를 이용하여 좀 더 다양한 정수 시퀀스를 생성할 수 있으며, 일반적인 형식은 다음과 같다.
- range(start, stop, step)이라고 호출하면 start에서 시작하여 (stop-1)까지 step 간격으로 정수들이 생성된다.
- 0, 1, 2, 3, 4가 바로 그것이다. 그리고 for 루프는 이 시퀀스의 갯 수만큼 반복 수행한다.
- 여기서 start와 step가 생략될 수 있는데 이 경우 start는 0으로 간주되고 step은 1로 간주된다.
- 하지만 **stop 값은 반드시 지정해야만 루프가 수행된다.**



range() 함수

- 예를 들어서 range(0, 5, 1)이라고 하면 0, 1, 2, 3, 4까지의 정수가 반환된다. range(5)라고 하면 start와 step은 생략된 것으로 range(0, 5, 1)와 같다.
- 만약 0이 아닌 1부터 시작하여서 5까지 반복하고 싶다면 어떻게 하면 될까? range(1, 6, 1)을 사용하면 될 것이다. 반복하면서 변수 i의 값을 출력하여 보자.

```
for i in range(1, 6, 1):  
    print(i, end = " ")    # end = " "로 지정하면 줄바꿈을 하지 않고 공백으로 나열한다.
```

1 2 3 4 5

- 만약 10부터 시작하여서 1까지 1씩 감소하며 반복하고 싶다면 어떻게 하면 될까? range(10, 0, -1)을 사용하면 될 것이다. 반복하면서 변수 i의 값을 출력하여 보자.

```
for i in range(10, 0, -1):  
    print(i, end = " ")
```

10 9 8 7 6 5 4 3 2 1

while 문

- 조건 제어 반복은 어떤 조건이 만족되는 동안 반복하기 때문에 붙여진 이름이다.
- 예를 들어서 자전거를 타고 가는데 앞에서 공사가 벌어져서 진행할 수가 없다. 공사가 진행중인 동안은 같은 구간을 계속해서 반복하는 것이다.
- 이 자전거는 한 바퀴 돌 때마다 공사가 끝났는지 확인을 한다. 공사중인 상태이면 반복을 지속하고 공사가 끝나면 탈출하는 것이다.

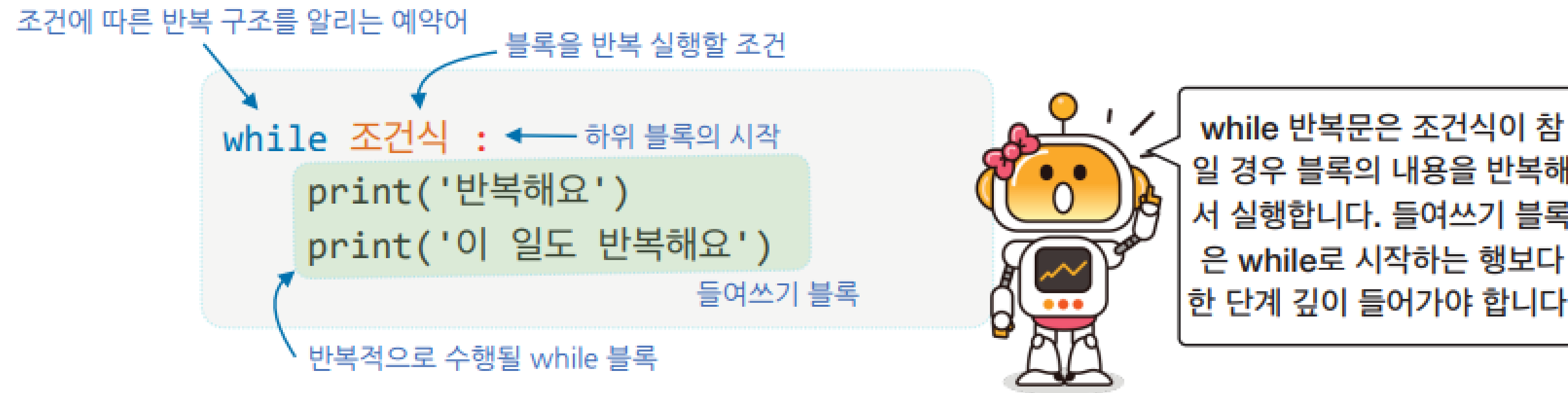
while 문

- 이런 경우에 조건 제어 반복을 사용한다. 조건 제어 반복은 while 반복문을 사용하는 것이 편리하다.



while 문

- while 반복문은 다음과 같은 구조를 갖는다.



위의 예를 프로그램으로 작성해보면 다음과 같다.

```
under_construction = True  
while under_construction:  
    response = input("공사가 완료 되었습니까?")  
    if response == "예" :  
        under_construction = False  
  
print("루프를 탈출했습니다.")
```

while 문

- 횟수를 알 수 없는 경우에만 while 루프를 사용할 수 있는 것은 아니다
 - 횟수를 알고 있는 경우에도 while 루프 가능
 - 예를 들어서 1부터 10까지의 합을 계산하는 코드를 while 루프로 작성해보자
 - 이를 위하여 1부터 10까지 증가하는 변수 count를 1로 초기화하여 만들자
 - 그리고 이 값들을 저장할 변수 s를 선언하고 0으로 초기화하자
 - 이 count에 s를 누적해서 더하는 코드를 다음과 같이 작성할 수 있을 것이다

```
count = 1
s = 0      # s는 누적하여 더한 값을 담을 변수로 0으로 초기화 함
while count <= 10 :
    s = s + count      # 매번 count 값을 s에 더함
    count = count + 1  # 매번 count 값을 1씩 증가시킴
print("합계는", s)
```

합계는 55

무한 루프와 break

- 조건 제어 루프에서 가끔은 프로그램이 무한히 반복하는 일이 발생한다. 이것은 **무한 루프** *infinite loop*라고 한다.
- 무한 반복이 발생하면 프로그램은 빠져 나올 수 없기 때문에 문제가 된다.
- 하지만 가끔은 무한 루프가 사용되는데 예를 들면 신호등 제어 프로그램은 무한 반복하여야 하기 때문이다.
- 무한 반복 루프는 다음과 같은 형태를 가진다.



무한 루프와 break

- while 루프의 조건에 True가 있다. 따라서 조건이 항상 참이므로 무한히 반복된다.
 - 무한 루프라고 하더라도 어떤 조건이 성립하면 무한 루프를 빠져나와야 하는 경우도 많다.
 - 이런 경우는 if 문장을 사용하여 루프를 빠져나오게 된다. break 문장은 루프를 강제로 빠져 나올 때 사용하는 문장이다.

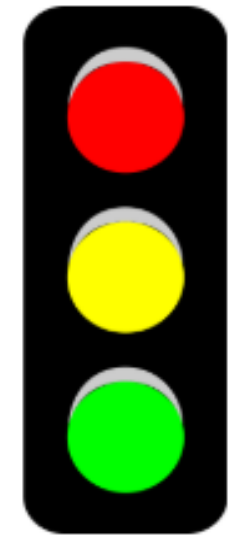
```
while True:
    light = input('신호등 색상을 입력하시오:')
    if light == 'green':
        break

print('녹색 신호입니다. 전진하세요!')
```

무한 루프와 break

- 위의 코드에서는 화면에 '신호등 색상을 입력하시오:'을 출력하고 사용자의 입력을 기다린다.
- while True:로 되어 있으므로 무한 루프이다.
- 사용자가 'green'를 입력하면 break 문장을 실행하여서 무한 루프를 빠져나간다.
- 사용자가 입력한 색상이 'green'이 아니면 이 루프는 계속 반복한다.

```
신호등 색상을 입력하시오: red
신호등 색상을 입력하시오: yellow
신호등 색상을 입력하시오: green
녹색 신호입니다. 전진하세요!
```



continue와 break

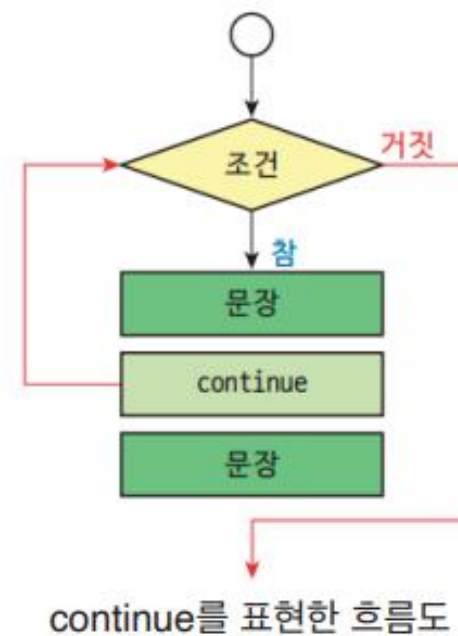
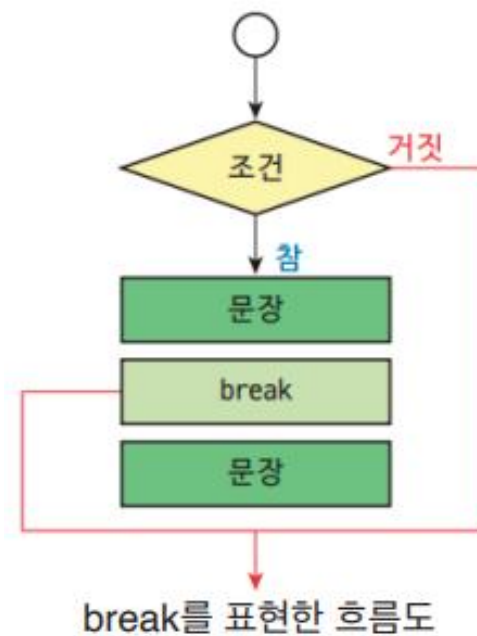
- 루프를 제어하는 첫번째 방법은 break 문으로 이 문장을 만나면 가장 가까운 블록을 즉시 빠져나오게 된다.
- 그렇다면 continue는 어떻게 수행될까? 이 키워드는 루프를 빠져나오지 않고 continue 아래의 문장만을 건너뛰는 역할을 한다.

continue와 break

- 즉, 반복문이 종료되는 것은 조건이 거짓일 때에만 해당한다. continue를 사용하는 예제코드가 아래에 있다.

```
st = 'I love Python Programming' # 출력을 위한 문자열
for ch in st:
    if ch in ['a','e','i','o','u', 'A','E','I','O','U']:
        continue # 모음일 경우 아래 출력을 건너뛴다
    print(ch, end='')
```

Iv Pythn Prgrmmng



break 문은 루프를 즉시 빠져나가지만, continue 문은 루프를 빠져나가지 않고 continue 아래쪽 문장을 건너뛰기만 합니다.

continue와 break



잠깐 – continue와 break를 너무 많이 쓰면 코드를 읽기 어려워진다

break와 continue는 프로그램의 제어를 효율적으로 하는데 편리하게 사용할 수 있다. 하지만, break와 continue문이 너무 많이 사용되는 경우 제어의 흐름에 일관성이 없어 프로그램을 이해하는 것이 어려워진다. 따라서 continue와 break는 필요한 경우에만 제한적으로 사용하는 것이 좋다.

리스트와 튜플

리스트는 무엇이고 왜 필요한가

- 여러 개의 데이터를 하나로 묶어서 저장하는 것이 필요하다. 파이썬에서는 **리스트** `list`와 **딕셔너리** `dictionary`를 이용하여 여러 개의 데이터를 한꺼번에 저장하고 처리할 수 있다.
- 이제까지 우리는 변수를 사용하여 데이터를 저장하고 처리하였다. 예를 들어서 어떤 사람의 키는 다음과 같이 저장할 수 있다.

```
>>> height = 178.9    # float 타입의 데이터를 저장한다.
```

- 수집한 데이터를 저장하기 위하여 변수를 하나씩 만들어주는 일은 번거롭기 짝이 없다. 데이터 과학자들은 많은 데이터를 처리하여야 한다. 좋은 방법이 있을까?

리스트는 무엇이고 왜 필요한가

사람이 100명이면
변수도 100개,
사람이 1,000명이면
???????

```
>>> height1 = 178.9    # float 타입의 데이터를 저장한다.  
>>> height2 = 173.5    # float 타입의 데이터를 저장한다.  
>>> height3 = 166.1    # float 타입의 데이터를 저장한다.  
>>> height4 = 164.3    # float 타입의 데이터를 저장한다.  
>>> height5 = 176.4    # float 타입의 데이터를 저장한다.
```

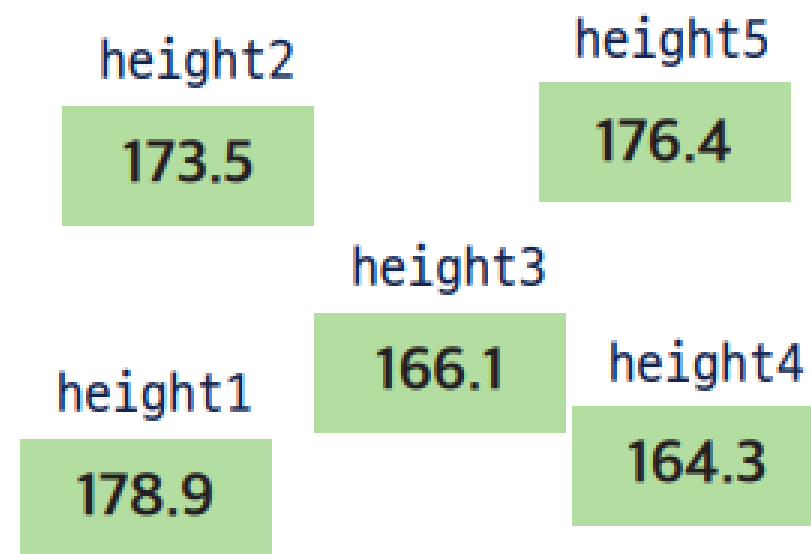


리스트는 무엇이고 왜 필요한가

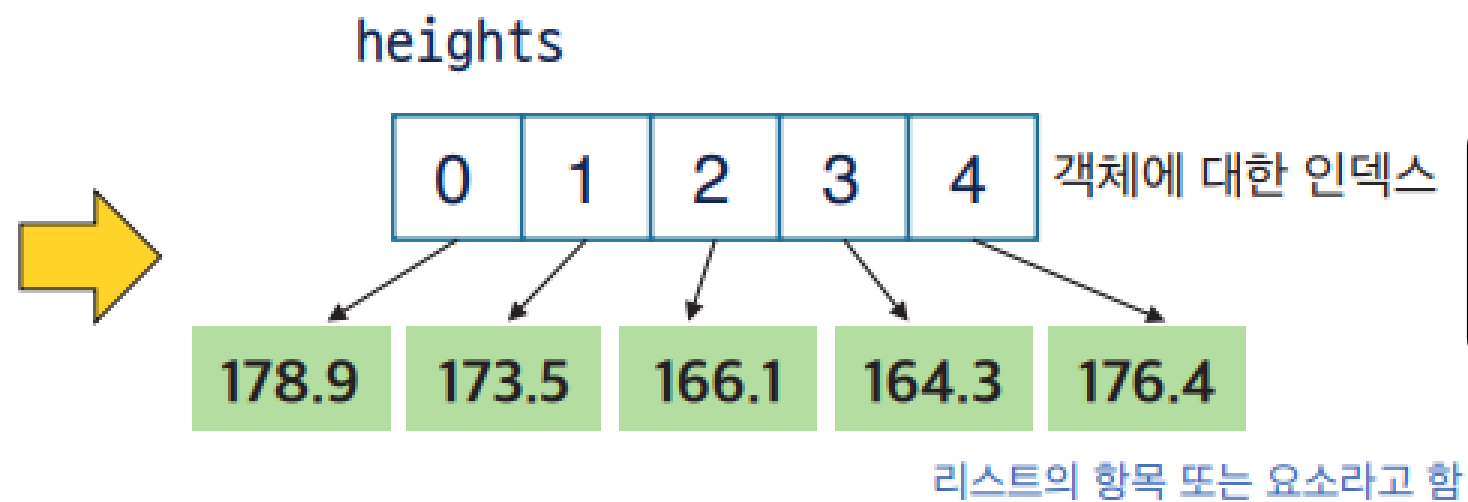
- 이럴 때는 "리스트(list)"를 만들고 여기에 데이터를 저장하면 된다. 파이썬에서 리스트는 대괄호를 이용해서 만들 수 있다. 예를 들어, 학생 5명의 키를 리스트에 저장하려면 다음과 같이 한다.

```
>>> heights = [178.9, 173.5, 166.1, 164.3, 176.4] # 리스트로 5명의 키를 저장
>>> heights
[178.9, 173.5, 166.1, 164.3, 176.4]
```

리스트를 사용하면
하나의 변수에 여러
개의 값을 담을 수 있다.



개별 변수로 표현된 데이터



리스트로 표현된 데이터와 인덱스

리스트 내의 개별 데이터를
항목 또는 요소라고 한다.

리스트를 생성하는 여러 가지 방법

- 파이썬에서 리스트는 다른 변수처럼 생성할 수 있다. 즉, 항목들을 [] 안에 적어주고 변수에 저장하면 리스트 변수가 생성된다. 다음과 같은 아이돌 그룹 BTS의 멤버 3명으로 리스트를 간단하게 만들어 보자.

```
>>> bts = ['V', 'Jungkook', 'Jimin']
```

- 만일 초기에 멤버가 없을 경우 다음과 같은 방식도 가능하다. 이때는 빈 리스트가 만들어 진다.

```
>>> bts = []  
>>> bts = list()
```

리스트를 생성하는 여러 가지 방법

```
>>> bts.append('Jin')
>>> bts.append('Suga')
>>> bts
['V', 'Jin', 'Suga']
```

리스트 내에 새 항목을
추가한다.

```
>>> bts = ['V', 'Jin', 'Suga', 'Jungkook']
>>> bts = bts + ['RM'] # 덧셈 연산자로 멤버 'RM'을 추가함
>>> bts
['V', 'Jin', 'Suga', 'Jungkook', 'RM']
```

리스트 내에 새 항목
'RM'을 추가한다.

```
>>> list(range(5))           # 0에서 5 미만 정수열을 생성(5는 포함 안됨)
[0, 1, 2, 3, 4]
>>> list(range(0, 5))        # list(range(5))와 동일한 결과
[0, 1, 2, 3, 4]
>>> list(range(0, 5, 1))      # list(range(0, 5))와 동일한 결과
[0, 1, 2, 3, 4]
>>> list(range(0, 5, 2))      # 생성하는 값을 2씩 증가시킴
[0, 2, 4]
>>> list(range(2, 5))         # 2에서 5미만의 연속된 수 2, 3, 4를 생성
[2, 3, 4]
>>> list(range(2, 6))         # 2에서 6미만의 연속된 수 2, 3, 4, 5를 생성
[2, 3, 4, 5]
>>> list('python')           # 'python' 문자열 시퀀스로부터 리스트를 생성
['p', 'y', 't', 'h', 'o', 'n']
```

range() 함수를 사용해서
여러 항목을 한꺼번에
생성할 수 있다.

리스트 연산을 해보자

- 파이썬에서는 리스트에 대해서도 다양한 연산이 가능하다. 예를 들어서 우리는 덧셈 연산으로 다음과 같이 두 개의 리스트를 합칠 수 있다.

```
>>> bts = ['V', 'J-Hope'] + ['RM', 'Jungkook', 'Jin']
>>> bts
['V', 'J-Hope', 'RM', 'Jungkook', 'Jin']
```

- 정수 리스트를 서로 더했을 때는 대응되는 원소끼리 더해지는 것이 아니라 리스트가 합쳐진다는 것을 명심하자. 대응되는 원소끼리 더하려면 뒤에서 설명하는 **넘파이NumPy**를 사용하여야 한다.

```
>>> num_list = [0, 1, 2] * 3    # [0, 1, 2]가 3회 반복되어 저장됨
>>> num_list
[0, 1, 2, 0, 1, 2, 0, 1, 2]
```

리스트에 사용 가능한 함수를 알아보자

- 필요에 따라서는 다음과 같이 리스트를 리스트 내부에 포함시키는 것도 가능하다. 예를 들어서 학생들의 이름과 키를 묶어서 리스트로 만들고 이것들을 모아서 하나의 리스트로 만들 수도 있다.

```
>>> student_list = [ ['Kim', 178.9], ['Park', 173.5], ['Lee', 176.1] ]  
>>> student_list  
[ ['Kim', 178.9], ['Park', 173.5], ['Lee', 176.1] ]
```

- 위의 student_list는 3개의 리스트를 포함하고 있다. 이 객체의 자료형을 알아보려면 type() 함수를 사용한다. student_list에 type()을 적용하면 다음과 같이 그 자료형 'list'가 출력된다.

```
>>> type(student_list)  
<class 'list'>
```


리스트에 사용 가능한 함수를 알아보자

- 리스트에 다음과 같은 수치값이 있을 경우 `len()`, `max()`, `min()`, `sum()`, `any()` 등의 내장함수를 사용하여 편리하게 활용할 수 있다. `len()`은 리스트의 길이를 반환하며, `max()`는 리스트 항목 중 최대값, `min()`은 리스트 항목 중 최소값을 반환한다.

```
>>> n_list = [200, 700, 500, 300, 400]
>>> len(n_list)      # 리스트의 길이를 반환한다
5
>>> max(n_list)      # 리스트 항목중 최대값을 반환한다
700
>>> min(n_list)      # 리스트 항목중 최소값을 반환한다
200
>>> sum(n_list)       # 리스트 항목의 합을 반환한다
2100
>>> sum(n_list) / len(n_list)  # 전체의 합을 원소의 개수로 나누면 평균값을 얻을 수 있다
420.0
>>> t_list = list((10, 20, 30)) # 튜플을 리스트로 변환
>>> t_list
[10, 20, 30]
>>> t_list.append(40)      # 새로운 값 40을 리스트 원소로 추가
>>> t_list
[10, 20, 30, 40]
```

리스트에 사용 가능한 함수를 알아보자

- 파이썬 내장함수인 `any()` 함수는 리스트에 참인 원소가 하나라도 있을 경우 `True`를 반환한다. 따라서 0과 ""(공백 문자열)만으로 이루어진 리스트는 `any()` 함수가 `False`를 반환한다. 0과 ""(공백 문자열)은 내부적으로 모두 `False`로 간주한다. 반면 `all()` 함수는 **리스트의 모든 원소가 참인 경우에만 `True`를 반환**한다.

```
>>> a_list = [10, 20, 30]      # 0이 아닌 숫자로 된 리스트
>>> b_list = [0, 10, 20, 30]   # 0과 0이 아닌 숫자로 된 리스트
>>> c_list = [0, '']          # 0과 빈 문자열을 가지는 리스트
>>> any(a_list), any(b_list), any(c_list) # 리스트내에 참인 원소 값이 존재하는가
(True, True, False)
>>> all(a_list), all(b_list), all(c_list) # 리스트내의 모든 원소가 참 값인가
(True, False, False)
```

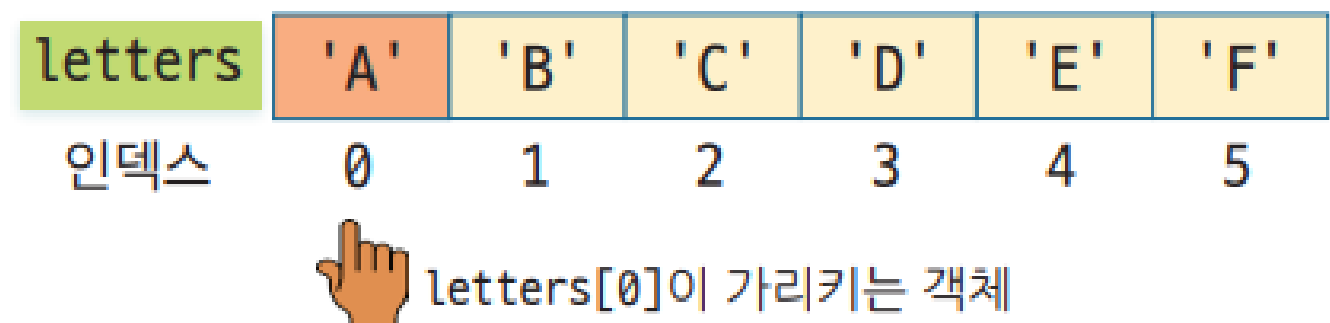
인덱싱과 슬라이싱을 해보자

- 파이썬 리스트에 저장된 데이터를 꺼내려면 어떻게 하면 될까? 예를 들어서 다음과 같이 알파벳 문자를 저장하고 있는 리스트가 있다고 하자.

```
>>> letters = ['A', 'B', 'C', 'D', 'E', 'F']
```

- 리스트에서 데이터를 추출하려면 **인덱스**index라는 정수를 사용한다. 리스트의 첫 번째 데이터는 인덱스 0을, 두 번째 요소는 인덱스 1을 가지게 된다. 나머지 요소들도 유사하게 인덱스가 할당된다.

```
>>> letters[0] # 리스트의 첫 번째 항목에 접근  
'A'
```



인덱싱과 슬라이싱을 해보자

```
>>> letters[1] # 리스트의 두 번째 항목에 접근  
'B'  
>>> letters[2] # 리스트의 세 번째 항목에 접근  
'C'
```

파이썬에서는 인덱스를
음수로 줄 수도 있다!
리스트의 맨 끝에 있는
데이터를 얻고자 할 때 유용

```
>>> letters[-1] # 리스트의 마지막 항목에 접근  
'F'
```

letters	'A'	'B'	'C'	'D'	'E'	'F'
인덱스	0	1	2	3	4	5
음수 인덱스	-6	-5	-4	-3	-2	-1



letters[-1]

인덱싱과 슬라이싱을 해보자

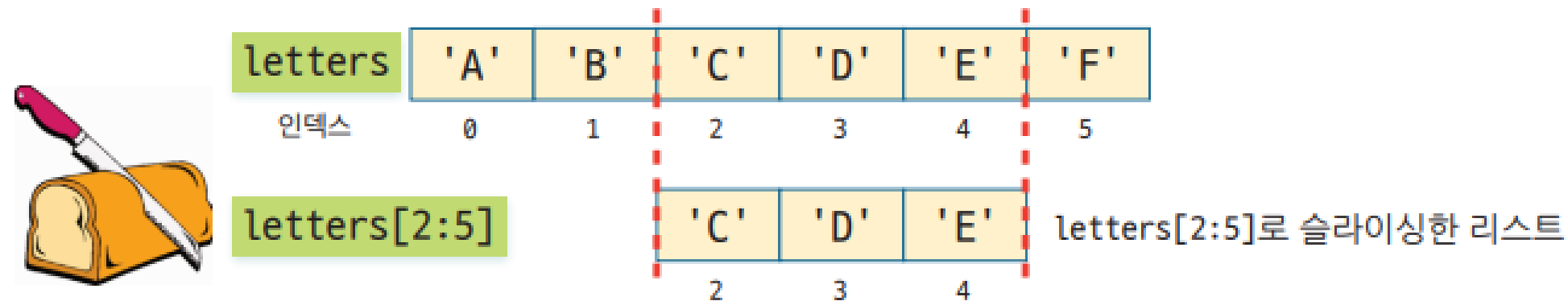
- 다음은 BTS 멤버의 영문 이름으로 이루어진 리스트이다. 이 예제는 리스트에 대하여 인덱싱을 이용하여 항목을 접근하거나 `min()`, `max()` 등의 함수를 통해서 항목을 가져오는 기능이 실려있다.

```
>>> bts = ['V', 'RM', 'Jungkook', 'J-Hop', 'Suga', 'Jin', 'Jimin']
>>> len(bts)                # 리스트의 원소 개수를 얻는다
7
>>> bts[len(bts)-1]         # 마지막 원소
'Jimin'
>>> bts[-1]                 # 음수 인덱스로 마지막 원소를 쉽게 접근할 수 있다
'Jimin'
```

리스트를 원하는 모양으로 자르는 슬라이싱

- 리스트에서 여러 요소를 선택해서 새로운 리스트를 만들고 싶다면, **슬라이싱 slicing**이라고 하는 기능을 사용할 수 있다. 리스트를 슬라이싱 하려면 다음과 같이 콜론을 이용하여 범위를 지정한다.

```
>>> letters = ['A', 'B', 'C', 'D', 'E', 'F']
>>> letters[2:5]
['C', 'D', 'E']
```



- 슬라이싱을 할 때 letters[2:5]와 같이 적으면 2부터 시작하여서 항목들을 추출하다가 5가 나오기 전에 중지한다.

리스트를 원하는 모양으로 자르는 슬라이싱

- 리스트를 슬라이싱하면 원래의 리스트는 손상되지 않으며, 새로운 리스트가 생성되어서 우리에게 반환된다. 즉, 슬라이스는 원래의 리스트의 부분 복사본이다.

```
>>> letters[:3]      # 리스트의 처음부터 세번째 항목까지 슬라이싱  
['A', 'B', 'C']
```

- 또한 두 번째 인덱스가 생략되면 리스트의 끝까지라고 가정한다. 이것은 생각보다 편리한 기능이다.

```
>>> letters[3:]      # 리스트의 네번째 항목부터 끝까지 슬라이싱  
['D', 'E', 'F']
```

- 극단적인 표현도 가능하다. 콜론만 있으면 리스트의 처음부터 끝까지라고 생각한다.

리스트를 원하는 모양으로 자르는 슬라이싱

```
>>> letters[:]
['A', 'B', 'C', 'D', 'E', 'F']
```

콜론만 있으면 리스트의 처음부터 끝까지

```
>>> letters[::2]
['A', 'C', 'E']
>>> letters[::-1]      # 뒤에서부터 앞으로 읽어오며 원소를 생성한다
['F', 'E', 'D', 'C', 'B', 'A']
```

리스트를 처음부터 끝까지 읽어오되, 스텝(step) 만큼 건너뛰기

리스트의 원소 값을 자유롭게 조작해 보자

- 우리는 리스트의 요소들을 조작할 수 있다. 리스트 요소 변경은 간단하다. 원하는 요소를 인덱스를 이용하여 지정한 후에 할당연산자로 새 값을 할당하면 된다.

```
>>> slist = ['Kim', 178.9, 'Park', 173.5, 'Lee', 176.1]
```

- 리스트의 슬라이싱을 사용하여 한 번에 여러 원소의 값을 변경할 수도 있다. 예를 들어서 Park의 이름을 'Paik'으로, 키를 180.0으로 바꿔주고 싶은 경우, slist[2:4]를 써준 뒤 이름과 키를 리스트로 적으면 된다.

```
>>> slist[2:4] = ['Paik', 180.0] # 슬라이싱 범위와 대치할 데이터
>>> slist
['Kim', 178.9, 'Paik', 180.0, 'Lee', 176.1]
```

리스트의 원소 값을 자유롭게 조작해 보자

```
>>> slist[3] = 175.0
```

'Park'의 키를 175.0로 변경하고 싶으면
인덱스를 사용하여 다음과 같이 수정

```
>>> slist  
['Kim', 178.9, 'Park', 175.0, 'Lee', 176.1]
```

함수를 사용하여 동적으로 항목을 추가하려면
append()나 insert() 함수를 사용할 수 있다

```
>>> slist.insert(4, 'Hong')  
>>> slist.insert(5, 168.1)  
>>> slist  
['Kim', 178.9, 'Paik', 180.0, Hong, 168.1, 'Lee', 176.1,]
```

리스트의 원소 값을 자유롭게 조작해 보자

- 리스트의 메소드와 하는 일을 정리하면 다음 표와 같다.

메소드	하는 일
<code>index(x)</code>	원소 <code>x</code> 를 이용하여 위치를 찾는다. 원소 <code>x</code> 의 인덱스 값을 반환한다.
<code>append(x)</code>	원소 <code>x</code> 를 리스트의 끝에 추가한다.
<code>count(x)</code>	리스트 내에서 <code>x</code> 원소의 개수를 반환한다.
<code>extend([x1, x2])</code>	<code>[x1, x2]</code> 리스트를 기존 리스트에 삽입한다.
<code>insert(index, x)</code>	원하는 <code>index</code> 위치에 <code>x</code> 를 추가한다.
<code>remove(x)</code>	<code>x</code> 원소를 리스트에서 삭제한다.
<code>pop(index)</code>	<code>index</code> 위치의 원소를 삭제한 후 반환한다. 이때 <code>index</code> 는 생략될 수 있으며 이 경우 리스트의 마지막 원소를 삭제하고 이를 반환한다.
<code>sort()</code>	값을 오름차순 순서대로 정렬한다. 키워드 인자 <code>reverse=True</code> 이면 내림차순으로 정렬한다.
<code>reverse()</code>	리스트를 원래 원소들의 역순으로 만들어 준다.

리스트 함축

- 파이썬은 **리스트 함축** `list comprehension`이라는 기능을 지원한다. 영어 `comprehension`은 함축, 포함, 내포라는 의미를 가지는데, 이는 수학자들이 집합을 정의하는 것과 유사하다.

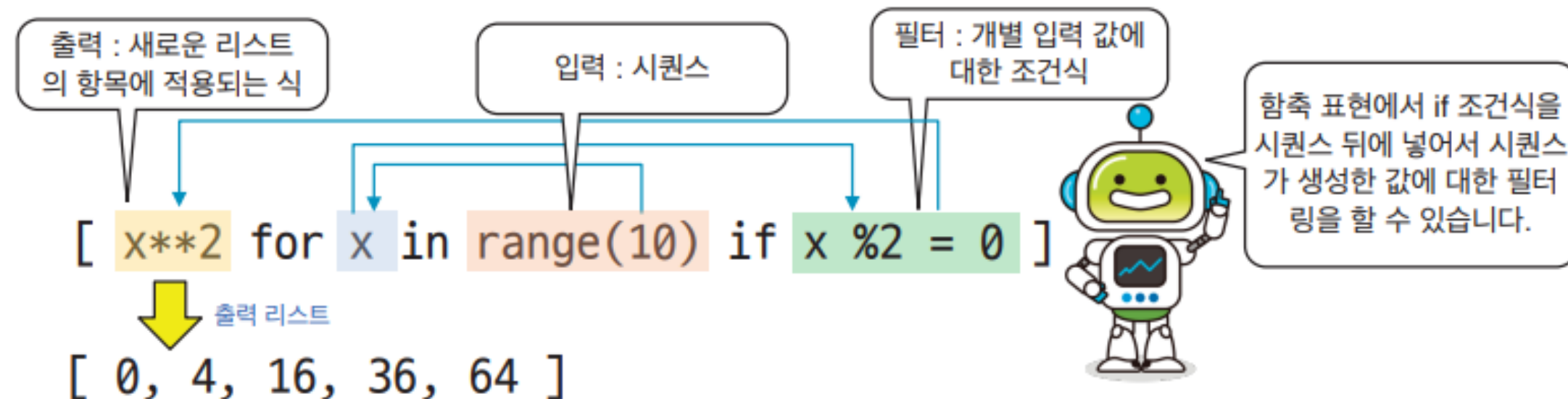
$$\{x^2 \mid x \in \mathbb{N}\}$$

```
>>> [ x**2 for x in [1, 2, 3, 4, 5] ] # 숫자 리스트에 대한 제곱으로 된 리스트  
[ 1, 4, 9, 16, 25 ]
```

조건이 붙는 리스트 함축표현도 가능하다

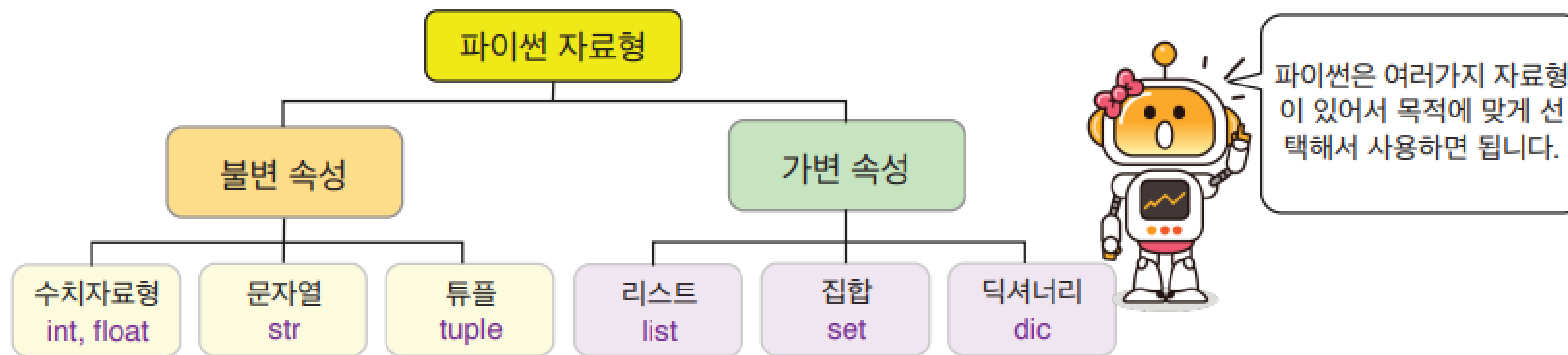
- 리스트 함축에는 `if`를 사용하여 조건이 추가될 수 있다. 예를 들어서 0부터 9 사이의 정수 중에서 짝수의 집합을 리스트 함축으로 표현하면 다음과 같다.
- 이때 `if x % 2 == 0`은 시퀀스가 생성한 값에 대하여 조건을 걸어주는 **필터 filter** 역할을 한다.

```
>>> [x for x in range(10) if x % 2 == 0]    # x % 2 필터를 통해서 출력식에 나타날 숫자를 결정
[0, 2, 4, 6, 8]
>>> [x**2 for x in range(10) if x % 2 == 0] # 출력 값에 제곱을 하자
[0, 4, 16, 36, 64]
```



한번 생성하면 그 값을 고칠 수 없는 자료형 : 튜플

- 파이썬의 데이터 형을 크게 분류하면 다음 그림과 같이 **불변속성**immutable 데이터 형, **가변속성**mutable 데이터 형으로 나눌 수 있다. **튜플**tuple은 리스트와 아주 유사하다. 하지만 튜플의 내용은 한 번 지정되면 변경될 수 없으며 대표적인 불변속성 자료형이다.
- 튜플은 한 번 그 내용이 정해지면 변경이 불가능하므로 구조가 단순하고 **리스트에 비하여 접근 속도가 빠르다**는 장점도 있다. 따라서 두 자료형은 목적에 따라서 적합한 경우에 선택하여 사용한다.



한번 생성하면 그 값을 고칠 수 없는 자료형 : 튜플

```
>>> colors = ('red', 'green', 'blue')  
>>> colors  
('red', 'green', 'blue')
```

색상을 저장하는 튜플

```
>>> numbers = (1, 2, 3, 4, 5)  
>>> numbers  
(1, 2, 3, 4, 5)
```

다수의 정수를 저장하는 튜플

한번 생성하면 그 값을 고칠 수 없는 자료형 : 튜플

- 다시 한번 강조하면 튜플은 **변경될 수 없는 객체** `immutable`이므로 튜플의 요소는 변경될 수 없다. 따라서 다음과 같이 **변경을 시도하면 에러**를 출력한다.

```
>>> t1 = (1, 2, 3, 4, 5)
>>> t1[0] = 100      # 에러가 출력됨
...
t1[0] = 100
TypeError: 'tuple' object does not support item assignment
```

튜플 자료형은 한번 생성하면 그 값을
고칠 수 없다 : 오류 발생

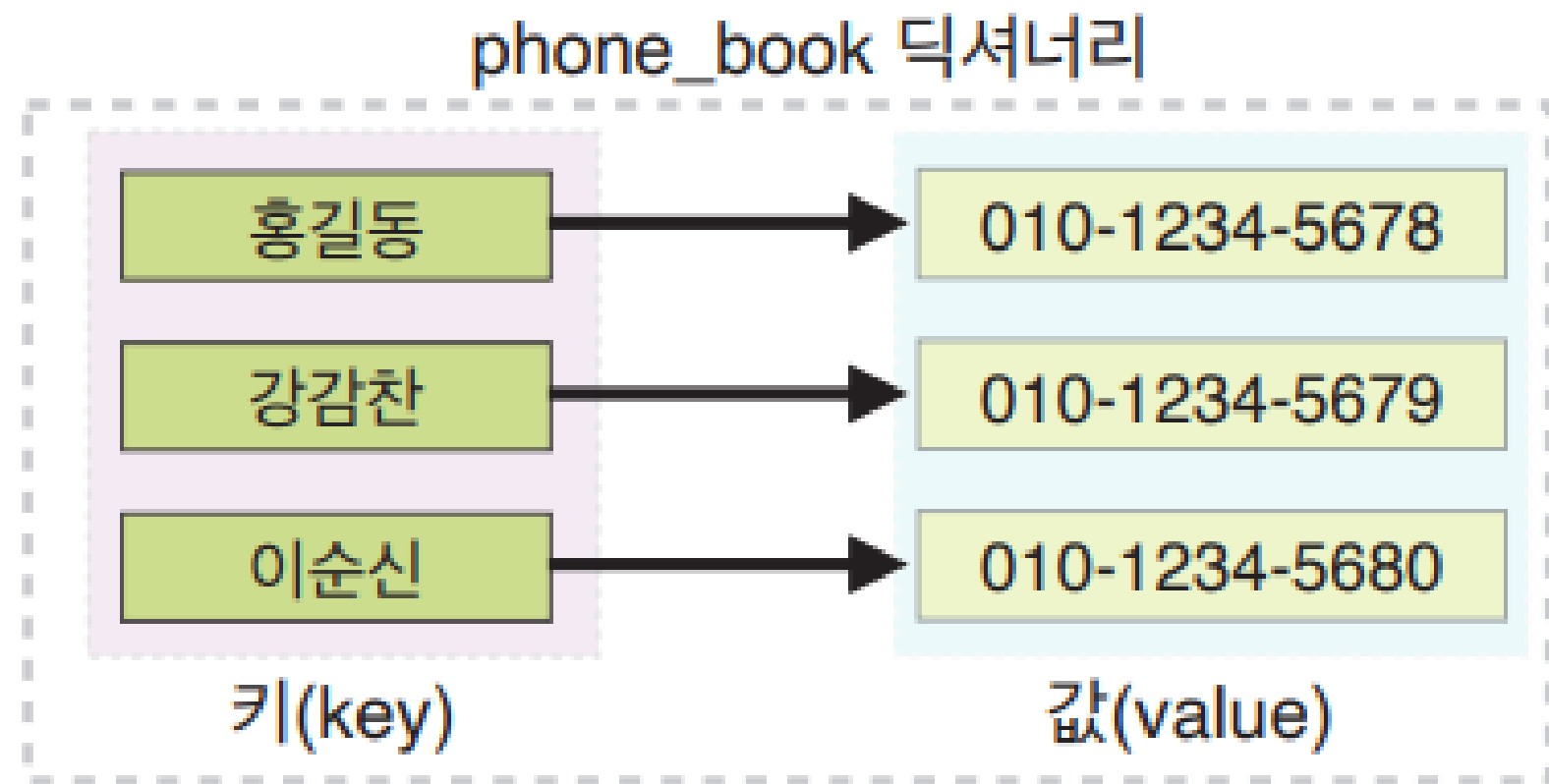
- 튜플도 시퀀스의 일종이기 때문에 인덱싱과 슬라이싱은 문자열이나 리스트와 동일하게 동작한다. 또한 튜플 간의 결합연산인 `+`와 반복연산자 `*`도 사용할 수 있다.

```
>>> t1 = (1, 2, 3, 4, 5)
>>> t1[0]          # 튜플의 인덱싱-리스트 인덱싱과 동일한 방식
1
>>> t1[1:4]        # 튜플의 슬라이싱 결과로 튜플을 반환함
(2, 3, 4)
>>> t2 = t1 + t1    # 튜플의 결합 연산
>>> t2
(1, 2, 3, 4, 5, 1, 2, 3, 4, 5)
```


딕셔너리와 집합

키와 값을 가진 딕셔너리

- **딕셔너리** dictionary도 리스트와 같이 값을 저장하는 자료구조로 파이썬에서는 기본 자료형으로 제공
- 하지만 딕셔너리에는 **값** value과 관련된 **키** key가 있다는 것이 큰 차이점



딕셔너리에는 값과 관련된 키가 있습니다. 예를 들어 핸드폰의 전화번호부가 있다고 할 때, 사람들의 이름이 키가 되고 전화번호가 그 키로 조회할 수 있는 값이 되지요.

키와 값을 가진 딕셔너리

- 파이썬의 딕셔너리에서는 서로 관련되어 있는 키와 값도 함께 저장되는데, 이것을 **키-값 쌍** key-value pair 이라 표현
- 딕셔너리를 만드는데는 몇 가지 방법이 있지만 일단 { }를 이용해서 공백 딕셔너리를 생성하고 추가 가능

```
>>> phone_book = { }    # 공백 딕셔너리를 생성
```

- 공백 리스트는 대괄호 []로 생성하고, 딕셔너리는 중괄호 { }로 생성한다는 것에 유의

```
>>> phone_book["홍길동"] = "010-1234-5678"
```


키와 값을 가진 딕셔너리

- 지금까지 작성한 내용 출력

```
>>> print(phone_book)
{'홍길동': '010-1234-5678'}
```

- phone_book 딕셔너리를 출력하면 딕셔너리의 항목item 이 쉼표로 구분되어 출력
 - 딕셔너리를 생성하면서 동시에 초기화도 가능

```
>>> phone_book = {"홍길동": "010-1234-5678"}
```

키와 값을 가진 딕셔너리

- 위 딕셔너리에 몇 개의 다른 전화번호를 추가해서 출력

```
>>> phone_book["강감찬"] = "010-1234-5679"  
>>> phone_book["이순신"] = "010-1234-5680"  
>>> print(phone_book)  
{'홍길동': '010-1234-5678', '강감찬': '010-1234-5679', '이순신': '010-1234-5680'}
```

딕셔너리의 기능을 알아보자

- 딕셔너리에서 가장 중요한 연산은 키를 가지고 연관된 값을 찾는 것
- 주소록이 있을 경우 사람 이름을 가지고 전화번호를 찾을 수 있어야함

```
>>> print(phone_book["강감찬"])  
010-1234-5679
```

- 리스트에서는 인덱스를 가지고 항목을 찾을 수 있지만 딕셔너리에서는 키가 있어야 값을 찾을 수 있음
- 딕셔너리에서 사용되는 모든 키를 출력하려면 다음과 같이 keys()라는 메소드를 사용

```
>>> phone_book.keys()  
dict_keys(['홍길동', '강감찬', '이순신'])
```

keys() 메소드를 사용하면 키를 얻을 수 있다.

딕셔너리의 기능을 알아보자

- 반면 딕셔너리에서 사용되는 모든 값을 출력하려면 values() 사용

```
>>> phone_book.values()
dict_values(['010-1234-5678', '010-1234-5679', '010-1234-5680'])
```

- 딕셔너리 내부의 모든 값을 출력하려면 items() 사용
- for문에서 phone_book.item()와 같은 함수를 호출하면 (키, 값) 튜플이 반환되

```
>>> phone_book.items()
dict_items([('홍길동', '010-1234-5678'), ('강감찬', '010-1234-5679'), ('이순신', '010-1234-5680')])
>>> for name, phone_num in phone_book.items():
...     print(name,':', phone_num)
...
홍길동 : 010-1234-5678
강감찬 : 010-1234-5679
이순신 : 010-1234-5680
```

딕셔너리의 항목들을
시퀀스로 추출할 수 있다.

딕셔너리와 리스트의 비교

- 리스트와 딕셔너리, 두 자료형이 가진 특성 중 가장 큰 차이를 보이는 인덱스와 키에 대한 비교

리스트	딕셔너리
첫 번째 값의 인덱스는 0으로 시작하며 이 인덱스 값은 자동으로 1씩 증가하며 할당된다.	사용자가 지정한 키와 값이 쌍을 이룬다.
예시: lst = [11, 22, 33, 44, 55]	예: dic = {0:11, 1:22, 2:33, 3:44, 4:55}
index 0 1 2 3 4 lst 11 22 33 44 55 lst[0] lst[1] lst[2] lst[3] lst[4]	dic key value 0 1 2 3 4 11 22 33 44 55 dic[0] dic[1] dic[2] dic[3] dic[4]

딕셔너리와 리스트의 비교

- 위의 lst를 생성한 후 pop(0) 메소드를 호출하면 아래 결과와 같이 가장 첫 번째 원소인 11이 반환되고 lst 객체의 목록에서 사라지는 것을 확인

```
>>> lst = [11, 22, 33, 44, 55]
>>> lst.pop(0)           # 리스트 항목의 첫 항목이 반환되고 삭제됨
>>> print('lst =', lst)
lst = [22, 33, 44, 55]
```

- dic이라는 딕셔너리를 생성한 후 pop(0) 메소드를 호출하면 아래 결과와 같이 키가 0인 첫 번째 원소인 {0:11}이 반환되고 dic 객체의 목록에서 사라짐

```
>>> dic = {0:11, 1:22, 2:33, 3:44, 4:55}
>>> dic.pop(0)           # 딕셔너리 항목중에서 키가 0인 항목 11이 반환되고 삭제됨
>>> print('dic =', dic)
dic = {1: 22, 2: 33, 3: 44, 4: 55}
```

딕셔너리와 리스트의 비교

- 리스트 객체인 lst와 딕셔너리 객체인 dic의 차이점

리스트	딕셔너리
lst.pop(0) 호출 후 lst : [22, 33, 44, 55]	dic.pop(0) 호출 후 dic : {1:22, 2:33, 3:44, 4:55}
index 01234 lst 1122334455 lst[0]lst[1]lst[2]lst[3]lst[4] ↓ lst.pop(0) 호출 후 index 0123 lst 22334455 lst[0]lst[1]lst[2]lst[3]	dic key 01234 value 1122334455 dic[0]dic[1]dic[2]dic[3]dic[4] ↓ dic.pop(0) 호출 후 dic key 1234 value 22334455 dic[1]dic[2]dic[3]dic[4]

딕셔너리와 리스트의 비교

- 아래 코드를 통해 차이점 확인

```
>>> print('lst[1] =', lst[1])  
      lst[1] = 33  
>>> print('dic[1] =', dic[1])  
      dic[1] = 22
```

- 실행 결과는 예상한 바와 같이 나타남
 - lst[0]를 출력하면 리스트의 첫 항목인 22가 출력
 - 반면 dic[0]를 출력하면 dic[0]에 해당하는 항목 11이 dic.pop(0)에 의해 삭제되었기 때문에, **오류 출력**

```
>>> print('lst[0] =', lst[0])    # 리스트의 첫 항목을 출력  
      lst[0] = 22  
>>> print('dic[0] =', dic[0])    # 주의 : 오류, 0을 키로 하는 항목이 없음  
      ....  
      KeyError: 0
```


항목의 수를 확인하고 삭제하기

- 리스트와 딕셔너리에서 모든 항목의 수를 구하는 방법, 그리고 특정한 요소 혹은 전체를 삭제하는 방법은 다음 표와 같이 큰 차이가 없음
 - 각각 개수를 확인하는 함수인 `len()`을 사용하거나, 지정된 요소를 삭제하는 명령인 `del`과, 요소들을 모두 삭제하는 메소드인 `clear()`을 사용하면 가능
- `del`은 파이썬의 객체를 삭제하는 명령 **

구분	리스트	딕셔너리
항목 수 확인	<code>len(lst)</code>	<code>len(dic)</code>
특정 항목 삭제	<code>del(lst[0])</code> 또는 <code>del lst[0]</code>	<code>del(dic[0])</code> 또는 <code>del dic[0]</code>
전부 삭제	<code>lst.clear()</code>	<code>dic.clear()</code>

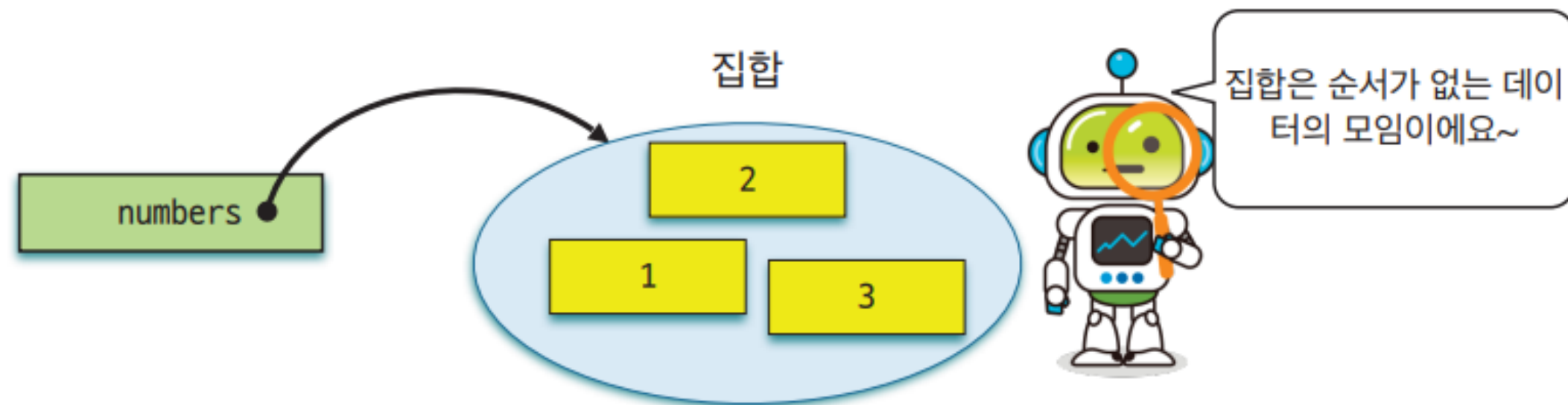
딕셔너리의 다양한 메소드

메소드	하는 일
<code>keys()</code>	딕셔너리 내의 모든 키를 반환한다.
<code>values()</code>	딕셔너리 내의 모든 값을 반환한다.
<code>items()</code>	딕셔너리 내의 모든 항목을 [키]:[값] 쌍으로 반환한다.
<code>get(key)</code>	키에 대한 값을 반환한다. 키가 없으면 <code>None</code> 을 반환한다.
<code>pop(key)</code>	키에 대한 값을 반환하고, 그 항목을 삭제한다. 키가 없으면 <code>KeyError</code> 예외를 발생시킨다.
<code>popitem()</code>	제일 마지막에 입력된 항목을 반환하고 그 항목을 삭제한다.
<code>clear()</code>	딕셔너리 내의 모든 항목을 삭제한다.

순서가 중요하지 않은 대상들, 집합

- 파이썬에서 사용하는 **집합**^{set} 역시 수학의 집합과 비슷하며, 튜플과 달리 순서가 없는 자료형
 - 동일한 값을 가지는 항목의 중복이 허용되지 않음
 - 교집합, 합집합, 차집합, 대칭차집합 등의 다양한 집합 연산을 수행 가능

```
>>> numbers = {2, 1, 3}    # 숫자 3개로 이루어진 집합 자료형
>>> numbers
{1, 2, 3}
```



순서가 중요하지 않은 대상들, 집합

- 리스트로부터 집합을 생성 가능
 - 집합자료형에서는 요소가 중복되면 자동으로 중복된 요소를 제거

```
>>> set([1, 2, 3, 1, 2])    # 리스트로부터 집합을 생성함
{1, 2, 3}
```

- 문자열로부터 집합을 생성하는 것도 가능 → 각 문자들이 하나의 요소

```
>>> set("abcdefa")         # 문자열도 시퀀스형이라서 집합형으로 변환이 가능하다
{'f', 'a', 'b', 'e', 'c', 'd'}
```

- 비어 있는 집합을 생성하려면 다음과 같이 set() 함수를 사용

```
>>> numbers = set()        # 비어있는 집합 생성
```

순서가 중요하지 않은 대상들, 집합

- 리스트의 경우 + 연산으로 두 리스트를 결합시키는 것이 가능하지만 집합은 이 연산을 지원하지 않음
 - {1, 2} + {1, 2, 3} 연산은 아래와 같이 TypeError

```
>>> n_list = [1, 2] + [1, 2, 3]
>>> print('n_list =', n_list)
n_list = [1, 2, 1, 2, 3]
>>> n_set = {1, 2} + {1, 2, 3}
....
TypeError: unsupported operand type(s) for +: 'set' and 'set'
```


집합의 항목에 접근하는 연산

- 어떤 항목이 집합 안에 있는지를 검사하려면 리스트와 마찬가지로 in 연산자 사용

```
numbers = {2, 1, 3}
if 1 in numbers:    # 1이라는 항목이 numbers 집합에 있는가 검사
    print("집합 안에 1이 있습니다.")
else:
    print("집합 안에 1이 없습니다.")
```

집합 안에 1이 있습니다.

- 집합의 항목은 순서가 없기 때문에 인덱스를 가지고 접근 불가능
- 하지만 for 반복문을 이용하여 각 항목들에 접근 가능

```
numbers = {2, 1, 3}
for x in numbers:
    print(x, end=" ")
```

1 2 3

집합의 항목에 접근하는 연산

- 단, 항목들이 출력되는 순서는 입력된 순서와 다를수도 있음
 - 만약 정렬된 순서로 항목을 출력하기를 원한다면 다음과 같이 `sorted()` 함수를 사용

```
for x in sorted(numbers):  
    print(x, end=" ")
```

```
1 2 3
```

집합의 항목에 접근하는 연산

- 집합의 요소에는 인덱스가 없기 때문에 인덱싱이나 슬라이싱 연산은 의미가 없음
- add() 메소드를 이용하여서 하나의 요소 추가 가능

```
>>> numbers = {1, 2, 3}
>>> numbers.add(4)      # numbers 집합에 원소 4를 추가
>>> numbers
{1, 2, 3, 4}
```

- 집합의 요소를 삭제할 때는 remove() 메소드 사용

```
>>> numbers.remove(4)   # numbers 집합에 원소 4를 삭제
>>> numbers
{1, 2, 3}
```


집합의 항목에 접근하는 연산



잠깐 - in 연산자는 집합에만 사용하는 것은 아니다

이 절에서 사용한 `in` 연산자는 집합에만 사용할 수 있는 연산자가 아니라 여러 항목을 가진 다양한 데이터에 적용할 수 있다. 대표적인 예로 리스트에 특정한 항목이 있는지도 다음과 같이 검사할 수 있다.

```
>>> a_list = ['hello', 'world', 'welcome', 'to', 'python']
>>> 'python' in a_list      # 'python' 문자열 항목이 a_list에 있는가 조사한다
True
```

집합 비교 연산

- 2개의 집합이 서로 같은 지 검사 시 ==과 != 연산자 사용

```
>>> A = {1, 2, 3}
>>> B = {1, 2, 3}
>>> A == B           # A가 B와 같은지 검사
True
```

- < 연산자와 <= 연산자를 사용하여 집합이 진부분집합인지, 부분집합인지를 검사가능

```
>>> A = {1, 2, 3, 4, 5}
>>> B = {1, 2, 3}
>>> B < A           # B가 A의 진부분집합인가 검사
True
```

항목들에 대한 정보 처리

- 집합에 대해서도 len(), max(), min(), sorted(), sum() 등의 메소드는 사용가능

```
>>> a_set = {1, 5, 4, 3, 7, 4 }    # 6개 항목으로 집합 생성
>>> len(a_set)                    # 항목의 개수는 중복을 제외하면 5개이다
5
>>> max(a_set)                    # 항목 가운데 가장 큰 수는 7
7
>>> min(a_set)                    # 항목 가운데 가장 작은 수는 1
1
>>> sorted(a_set)                 # 항목을 정렬하여 리스트 만든다. 집합이므로 중복은 제거한다
[1, 3, 4, 5, 7]
>>> sum(a_set)                    # 중복 원소는 하나만 사용되므로 전체 합은 20
20
```

집합에 적용할 수 있는 논리 연산

- 다수의 항목을 가진 데이터에 대해 적용할 수 있는 논리 연산 `all()`과 `any()` 함수 적용 가능
 - 인자로 주어진 데이터의 항목들에 대해 부울형 평가 한 결과 종합적으로 반환

```
>>> a_set = { 1, 0, 2, 3, 3}
>>> all(a_set)    # a_set이 모두 True인가를 검사한다
False
>>> any(a_set)    # a_set에 True인 것이 하나라도 있는가를 검사한다
True
```

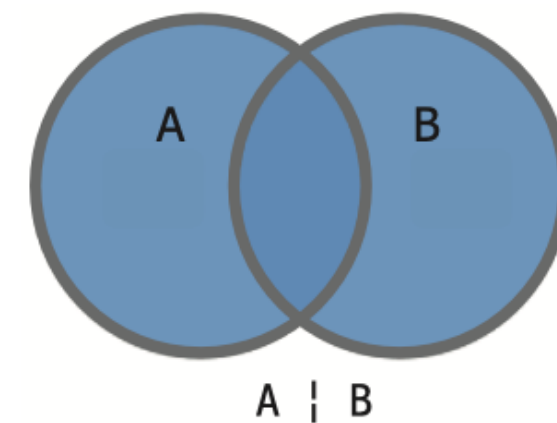
합집합, 교집합, 차집합, 대칭차집합 연산

- 집합이 유용한 이유 → 교집합이나 합집합과 같은 여러가지 집합 연산을 지원하기 때문

```
>>> A = {1, 2, 3}
>>> B = {3, 4, 5}
```

- 합집합은 다음과 같이 2개의 집합을 합하는 연산으로 | 연산자나 union() 메소드 사용

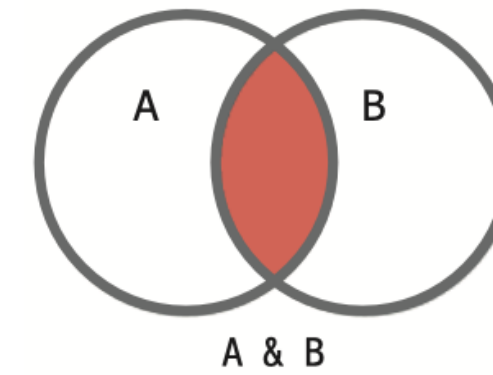
```
>>> A | B      # 합집합 연산
{1, 2, 3, 4, 5}
>>> A.union(B) # 합집합 메소드
{1, 2, 3, 4, 5}
```



합집합, 교집합, 차집합, 대칭차집합 연산

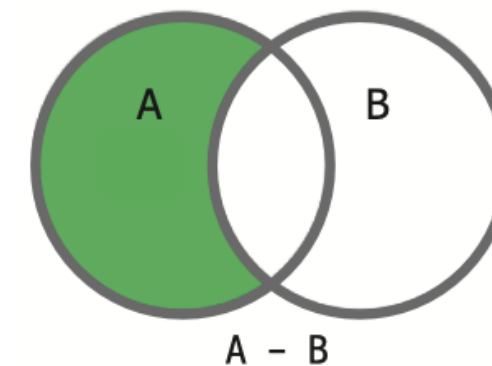
- 교집합은 `&` 연산자나 `intersection()` 메소드를 사용한다.

```
>>> A & B      # 교집합 연산
{3}
>>> A.intersection(B) # 교집합 메소드
{3}
```



- 차집합은 `-` 연산자나 `difference()` 메소드를 사용한다.

```
>>> A - B      # 차집합 연산
{1, 2}
>>> A.difference(B) # 차집합 메소드
{1, 2}
```



감사합니다

Jinu Gong